STUDY NOTES | CSE3015

AWS Cloud Practitioner

MODULE 3 — CO3 — PART 2

AWS Security, Identification & Compliance

AWS IAM	AWS ACM	AWS KMS
IDENTITY & ACCESS MANAGEMENT	CERTIFICATE MANAGER	KEY MANAGEMENT SERVICE

Concept-focused study notes | Definitions · Architecture · Diagrams · Comparisons · Examples
Not for exam prep — for deep conceptual understanding

Table of Contents

TOPIC 1 — AWS Identity and Access Management (IAM)

1.1 Definition & Purpose 3
1.2 Authentication vs Authorization 3
1.3 IAM Components Overview 4
1.4 Root User 4
1.5 IAM Users 5
1.6 IAM Groups 5
1.7 IAM Roles 6
1.8 IAM Policies — Structure & Types 7
1.9 Least Privilege Principle 9
1.10 Multi-Factor Authentication (MFA) 9
1.11 IAM Architecture 10

TOPIC 2 — AWS Certificate Manager (ACM)

2.1 Definition & Purpose 11
2.2 Digital Certificates, SSL, TLS, HTTPS 11
2.3 Certificate Types & Lifecycle 12
2.4 ACM Architecture & Request Flow 13
2.5 Real-World Example 13

TOPIC 3 — AWS Key Management Service (KMS)

3.1 Definition & Purpose 14
3.2 Encryption at Rest vs In Transit 14
3.3 KMS Keys — Types & Management 15
3.4 Symmetric Encryption 15
3.5 Asymmetric Encryption 16
3.6 Envelope Encryption 17
3.7 AWS Service Integration 18

COMPARISONS & ARCHITECTURE

Important Comparison Tables (15 tables) 19
Integrated Security Architecture 23
Final Concept Map 24

TOPIC 1 OF 3

AWS Identity and Access Management (IAM)

Control who can do what on your AWS account — securely and granularly

1.1 Definition & Purpose

DEFINITION

AWS IAM (Identity and Access Management) is a **global AWS service** that enables you to manage **who** (identity) can access AWS resources and **what** (access) they are allowed to do — without incurring additional cost.

Why IAM is Required

- AWS accounts can host dozens of services (EC2, S3, RDS, Lambda…)
- Multiple users (developers, admins, auditors) need different levels of access
- Without IAM, every user would need the root account credentials — a catastrophic security risk
- IAM provides **granular control**: each user/service gets only what it needs
- Supports compliance requirements (least privilege, audit trails)

Key Concepts Before Components

🪪 Identity
Who you are — a user, group, role, or service. The *subject* requesting access.

🔑 Access
What you can do — read, write, delete, execute. The *action* permitted on a resource.

✅ Permission
A granted right to perform a specific action on a specific resource.

🔐 Authentication
Verifying *who you are* — "Prove your identity."

🚦 Authorization
Determining *what you can do* — "You are allowed to do X."

📄 Policy
A JSON document defining what is allowed or denied on which resources.

1.2 Authentication vs Authorization

🔑 IMPORTANT DISTINCTION

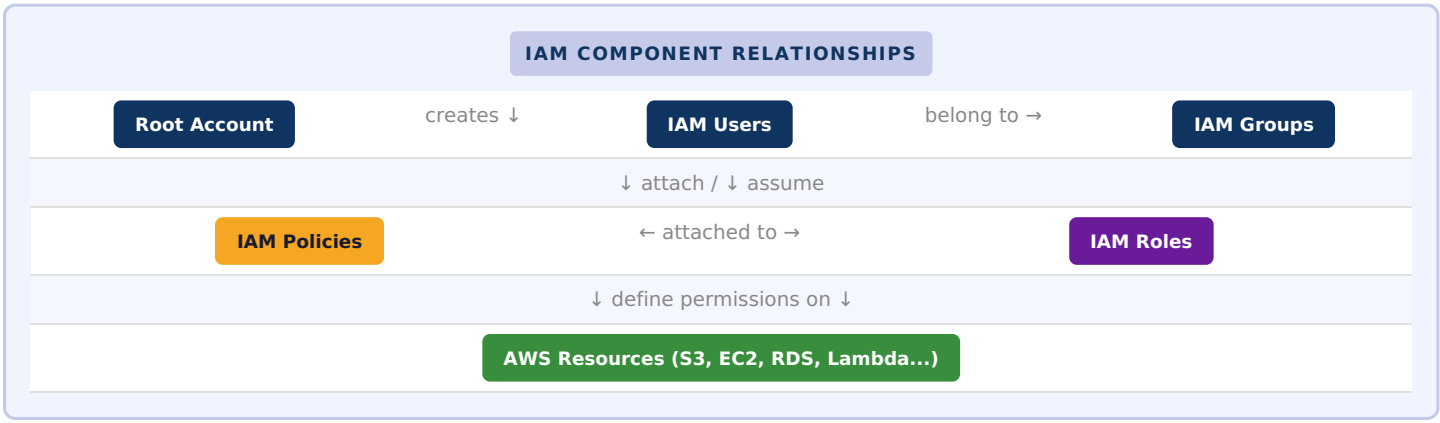
These two concepts are often confused but are fundamentally different steps in the security flow.

Aspect	Authentication	Authorization
Question	Who are you?	What can you do?
Purpose	Verify identity	Grant/deny access
How	Password, MFA, key pair	IAM policies & permissions
Happens	First (before authorization)	Second (after authentication)
IAM Element	Username + Password / Access Keys	Policies attached to identities
Failure result	Login rejected	Access denied (403)
Example	Alice logs in with her password ✓	Alice tries to delete S3 bucket — DENIED ✗

🌍 REAL-WORLD ANALOGY

- **AUTHENTICATION**
  = Showing your ID card at the office building entrance — security confirms you work there
- **AUTHORIZATION**
  = Your ID card only lets you access floors 1–3, not the server room on floor 5

1.3 IAM Components Overview



1.4 Root User

DEFINITION
The **root user** is the initial account owner identity created when an AWS account is first set up. It uses the email address and password used to create the account and has **unrestricted access to all AWS services and resources**.

Why Root Has Extensive Privileges	Security Best Practices
• It is the account's original owner identity • Can perform actions no other user can — e.g., close the account, change account settings • Can access all services with no permission restrictions • Acts as the "super administrator" of the AWS account	• **Never use root for day-to-day tasks** • Enable MFA on root immediately after account creation • Create IAM admin users for routine administration • Lock away root credentials (store securely, rarely accessed) • Root has no IAM permissions restrictions — extremely dangerous if compromised

⚠ **TASKS ONLY ROOT CAN PERFORM**
Close AWS account | Change account email / password | Enable MFA delete on S3 | Restore IAM user permissions | Activate IAM access to billing

1.5 IAM Users

DEFINITION
An **IAM User** is a persistent identity within an AWS account representing a specific *person or application*. It has long-term credentials (username/password or access keys) and is granted specific permissions.

What a User Represents

- A specific human (e.g., Alice the developer, Bob the DBA)
- An application or service running outside AWS needing programmatic access
- A system account for automation scripts

User Credentials — Two Types

Credential Type	Used For	Components

Console Password	AWS Management Console (web UI) login	Username + Password (+MFA)
Access Keys	Programmatic access (CLI, SDK, API)	Access Key ID + Secret Access Key

When IAM Users Are Appropriate

- Human users who need to sign into AWS Console
- Applications running outside AWS that need API access
- Cross-account access when roles aren't feasible
- **NOT recommended** for applications running on AWS services (use Roles instead)

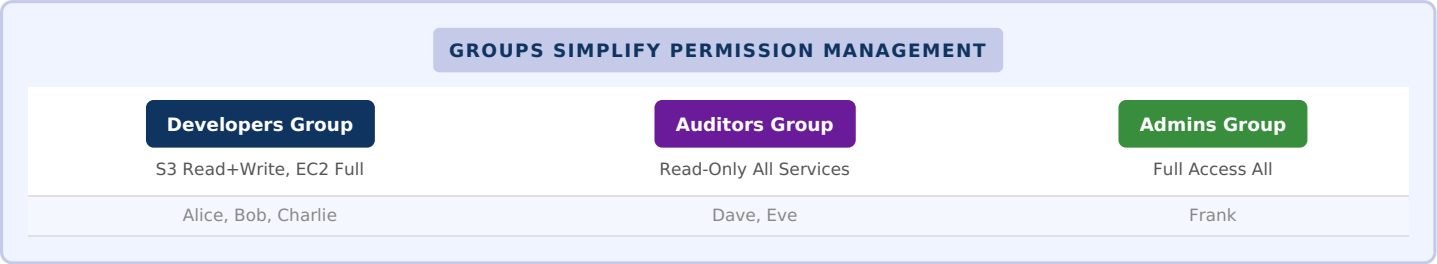
> 📌 **KEY RELATIONSHIP**
> An IAM User by itself has NO permissions. Permissions must be explicitly granted via policies — either directly or through group membership.

1.6 IAM Groups

DEFINITION
An **IAM Group** is a collection of IAM users. Policies attached to a group apply to *all users in that group*. Groups are used to manage permissions for multiple users efficiently.

Why Groups Simplify Administration

- Assign one policy to the group → all members inherit it instantly
- When a user's role changes, move them to a different group
- No need to update permissions for individual users one by one
- Easier to audit — look at one group to understand all its members' permissions



Important Rules About Groups

- A user can belong to **multiple groups**
- Groups cannot contain other groups (no nested groups)
- Groups have no credentials — they are purely a permission management mechanism
- Policies are attached to the group, not the individual user

1.7 IAM Roles

DEFINITION

An **IAM Role** is an identity with specific permissions that can be *assumed* by trusted entities (AWS services, users, or external identities). Unlike users, roles have **no long-term credentials** — they provide temporary security credentials when assumed.

Key Concepts of Roles

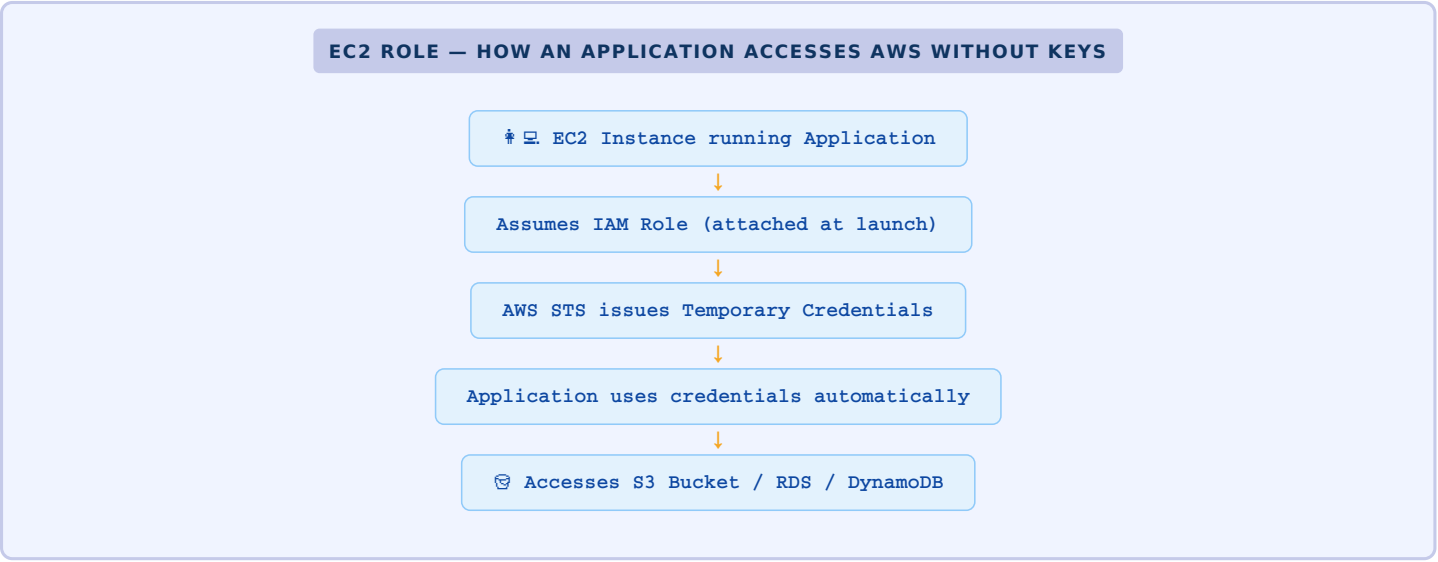
- **Trust Relationship:** Defines *who* is allowed to assume (use) the role — e.g., EC2 service, another AWS account, a Lambda function
- **Permissions Policy:** Defines *what* the role can do once assumed — same JSON policy format as user policies
- **Temporary Credentials:** When a role is assumed, AWS STS (Security Token Service) generates short-lived credentials (Access Key ID, Secret Access Key, Session Token) — they expire automatically
- **Role Assumption:** The act of "taking on" a role to gain its permissions temporarily

Common Role Types

Role Type	Who Assumes It	Use Case
EC2 Instance Role	EC2 instances	Allow an EC2 app to read from S3 without embedding access keys
Lambda Execution Role	Lambda functions	Allow Lambda to write logs to CloudWatch, read from DynamoDB
Cross-Account Role	Users in another AWS account	Allow Account B users to access resources in Account A
Service-Linked Role	AWS managed services	Created and managed by AWS for specific service integrations
Federated Role	External identity providers	Allow corporate directory (Active Directory) users to access AWS

Why Roles Are Preferred Over Embedding Credentials in AWS Services

- **NO STATIC CREDENTIALS:**
  Access keys embedded in code/config can be leaked in version control or logs
- **AUTO-ROTATION:**
  Temporary credentials expire automatically — no manual rotation needed
- **LEAST PRIVILEGE:**
  Roles grant only the permissions needed for the task
- **AUDITABILITY:**
  CloudTrail logs which role performed which action
- **NO SHARING:**
  Each service instance gets its own temporary credentials



1.8 IAM Policies — Structure & Types

DEFINITION

An **IAM Policy** is a JSON document that defines *permissions*. It specifies what actions are allowed or denied on which AWS resources, optionally under certain conditions.

Policy JSON Structure

```
{
  "Version": "2012-10-17",          // Policy language version (always use this)
  "Statement": [              // Array of permission rules
    {
      "Sid": "AllowS3ReadAccess",  // Optional: Statement ID (label)
      "Effect": "Allow",          // Allow OR Deny
      "Action": [              // What action(s) are permitted
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [            // Which AWS resource(s)
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ],
      "Condition": {           // Optional: extra conditions
        "StringEquals": {
          "aws:RequestedRegion": "us-east-1"
        }
      }
    }
  ]
}
```

Explaining Every Field

Field	Required	Values	Meaning
Version	Yes	"2012-10-17"	Policy language version — always use 2012-10-17
Statement	Yes	Array of rules	One or more permission rules in the policy
Sid	No	Any string	Statement identifier — a label for the rule
Effect	Yes	Allow / Deny	Whether to permit or block the action. Deny overrides Allow.
Action	Yes	service:operation	The AWS API action(s) — e.g., s3:GetObject, ec2:StartInstances
Resource	Yes	ARN or *	The specific AWS resource(s) the action applies to. * = all
Condition	No	Key-value pairs	Extra constraints — time, IP, MFA, region, tags

⚠ **EXPLICIT DENY ALWAYS WINS**

If multiple policies apply and one says Deny, the action is ALWAYS denied — regardless of any Allow statement elsewhere.

Default Deny

- In AWS IAM, everything is **denied by default**
- You must explicitly *Allow* an action for it to be permitted
- The permission evaluation order: **Explicit Deny → Explicit Allow → Default Deny**

Policy Types

Policy Type	Attached To	Who Creates/Manages	Portability
Identity-based	User, Group, Role	You (customer)	Follows the identity
Resource-based	AWS Resource (S3, KMS…)	You (customer)	Stays with the resource
AWS Managed	Any identity	AWS creates & maintains	Reusable across accounts
Customer Managed	Any identity	You create & maintain	Reusable within your account
Inline	Single user/group/role	You create & maintain	Embedded — not reusable
Permissions Boundary	User or Role	You (admin)	Sets maximum permission ceiling

Identity-based vs Resource-based Policy

Identity-Based Policy	Resource-Based Policy
• Attached to an IAM **user, group, or role** • Controls what that identity *can do* • Example: "Alice can read from any S3 bucket" • Most common type of IAM policy	• Attached directly to an **AWS resource** (e.g., S3 bucket policy) • Controls *who can access* that resource • Example: "This S3 bucket can only be accessed by Account 123456" • Can grant cross-account access without assuming a role

AWS Managed vs Customer Managed vs Inline Policy

Aspect	AWS Managed	Customer Managed	Inline
Creator	AWS	You	You
Management	AWS updates it	You maintain it	You maintain it
Reusability	Any AWS account	Within your account	Only 1 identity
Versioning	Automatic by AWS	Supported	Not supported
Best for	Common standard permissions	Custom fine-grained access	Strict 1:1 binding
Example	AmazonS3ReadOnlyAccess	MyAppS3Policy	Embedded in 1 role

1.9 Least Privilege Principle

DEFINITION

The **Principle of Least Privilege** states that every identity (user, role, service) should be granted *only the minimum permissions necessary* to perform its intended function — and nothing more.

Why It Matters

- **Limits blast radius:** If credentials are compromised, the attacker can only do what that identity was allowed to do
- **Reduces insider threats:** Employees can only access what they need for their job
- **Compliance:** Many security standards (PCI-DSS, HIPAA, SOC2) mandate least privilege
- **Better audit trail:** Narrow permissions make suspicious activity more visible

Example

SCENARIO: A LAMBDA FUNCTION PROCESSES IMAGES AND STORES THEM IN S3

- **Bad practice:** Grant Lambda the *AdministratorAccess* managed policy — it can now delete EC2 instances, modify IAM, etc.
- Good practice: Grant Lambda *only* `s3:PutObject` on *only* the specific bucket it writes to

1.10 Multi-Factor Authentication (MFA)

DEFINITION

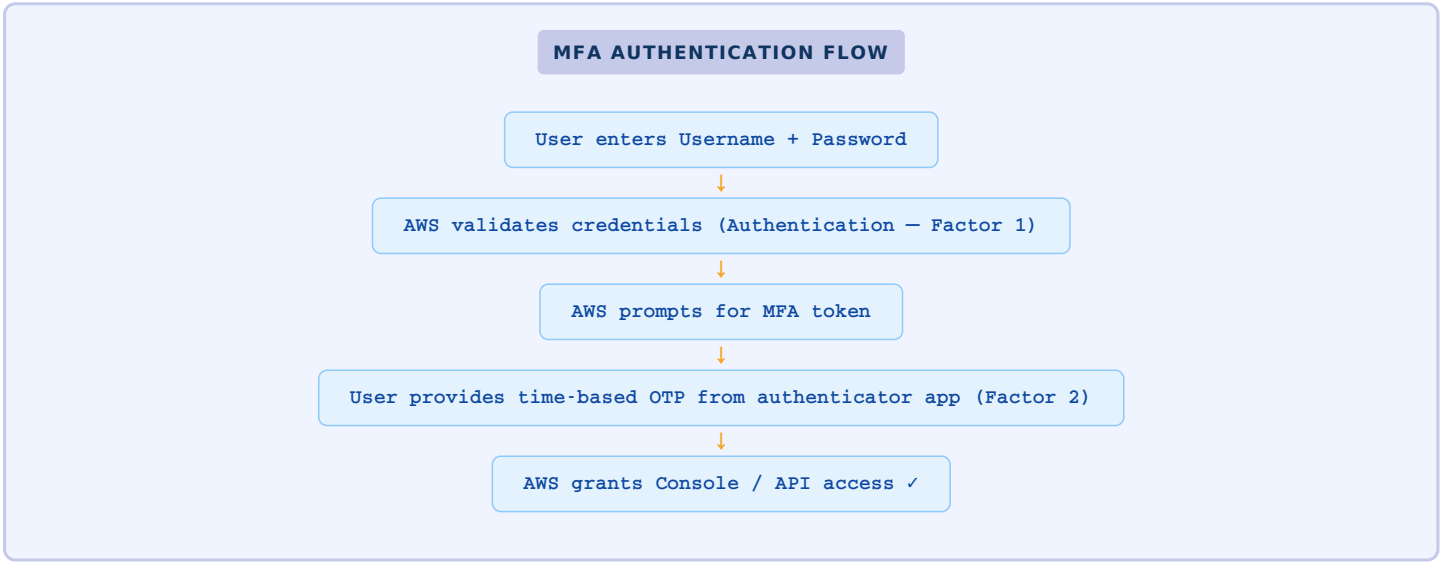
MFA adds an extra layer of security by requiring a user to provide *two or more verification factors* to authenticate — something they **know** (password) and something they **have** (authentication device/code).

MFA Authentication Factors

Factor Type	What It Is	Example
Something you **know**	Knowledge factor	Password, PIN
Something you **have**	Possession factor	Authenticator app, hardware token, OTP SMS
Something you **are**	Biometric factor	Fingerprint, face recognition

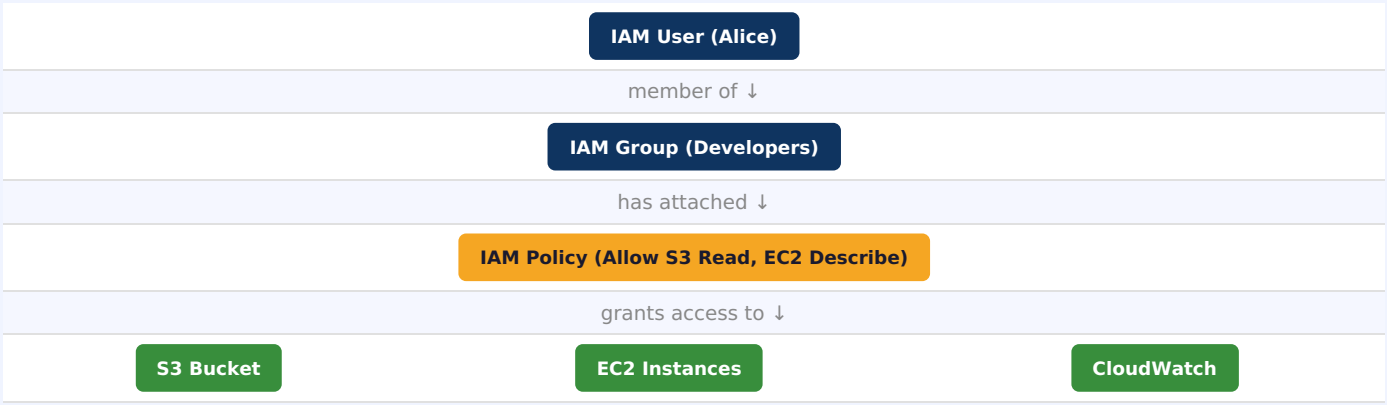
Why MFA Improves Security

- Password alone is vulnerable to phishing, brute force, data breaches
- Even if a password is stolen, the attacker still cannot log in without the second factor
- AWS strongly recommends MFA on root user and all privileged IAM users

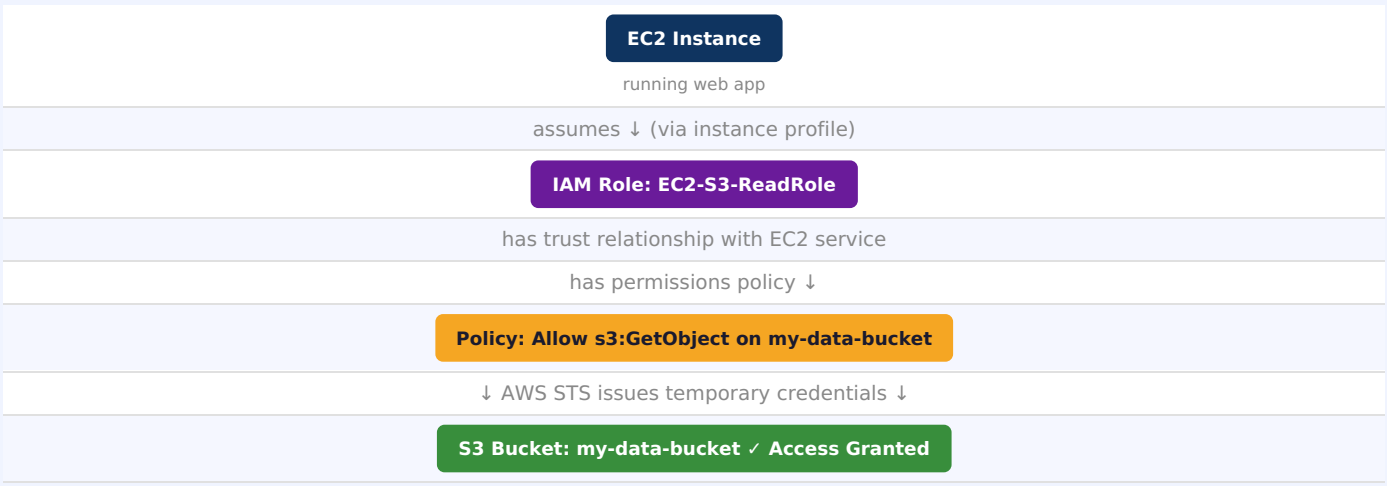


1.11 IAM Full Architecture

ARCHITECTURE 1 — USER → GROUP → POLICY → RESOURCE



ARCHITECTURE 2 — EC2 ROLE → SERVICE ACCESS (NO HARDCODED KEYS)



Important Relationships Summary

- **User ↔ Group:** A user can be in multiple groups; groups cannot be nested
- **Group → Policy:** Policy attached to group grants permissions to all members
- **Role ↔ Policy:** Roles have permission policies (what they can do) + trust policies (who can assume them)
- **Policy → Resource:** Policy's Resource field specifies the ARN of the target AWS resource
- **User ↔ Role:** Users can assume roles to temporarily gain different permissions

Terminology Quick Reference

ARN
Amazon Resource Name — globally unique identifier for any AWS resource. Format: arn:partition:service:region:account:resource

STS
Security Token Service — issues temporary credentials when a role is assumed

Principal
The entity (user, role, service) making the API request

Trust Policy
Part of a role definition that specifies who is allowed to assume it

Instance Profile
Container for a single IAM role that can be attached to an EC2 instance

Permission Boundary
Policy that sets the maximum permissions an identity can ever have

Commonly Confused IAM Concepts

Confusion	Clarification
Role vs User	Users have permanent credentials; Roles provide temporary credentials and are assumed by entities
Group vs Role	Groups are for collecting users; Roles are for assigning permissions to services/apps
Policy vs Permission	A Policy is the document; a Permission is the individual Allow/Deny rule within it
Authentication vs Authorization	Auth = "who you are"; Authz = "what you can do" — two separate steps
Inline vs Managed Policy	Inline is embedded and non-reusable; Managed is standalone and can be reused
Trust Policy vs Permissions Policy	Trust Policy = who can assume the role; Permissions Policy = what the role can do

TOPIC 2 OF 3

AWS Certificate Manager (ACM)

Provision, manage, and deploy SSL/TLS certificates — automatically and at no cost

2.1 Definition & Purpose

DEFINITION

AWS Certificate Manager (ACM) is a managed AWS service that handles the complexity of provisioning, managing, deploying, and renewing *SSL/TLS certificates* for use with AWS services like Elastic Load Balancing, CloudFront, and API Gateway.

Purpose

- Eliminate manual certificate management — no more buying, installing, or renewing certs manually
- Enable **HTTPS** (encrypted) communication between clients and AWS-hosted applications
- Provide **public** certificates (for internet-facing sites) at **no cost**
- Provide **private** certificates (for internal infrastructure) via ACM Private CA
- Auto-renewal prevents certificate expiry outages

2.2 Digital Certificates, SSL, TLS & HTTPS

Digital Certificate
An electronic credential that proves the identity of a server/entity and contains its public key. Issued by a trusted Certificate Authority (CA).

SSL (Secure Sockets Layer)
Older cryptographic protocol (now deprecated). Established the concept of encrypting web traffic between client and server.

TLS (Transport Layer Security)
The modern, more secure successor to SSL. The actual protocol used today for encrypting internet communication.

HTTPS
HTTP + TLS. The web protocol that uses TLS to encrypt HTTP traffic. URLs start with https:// and use port 443.

HTTP vs HTTPS

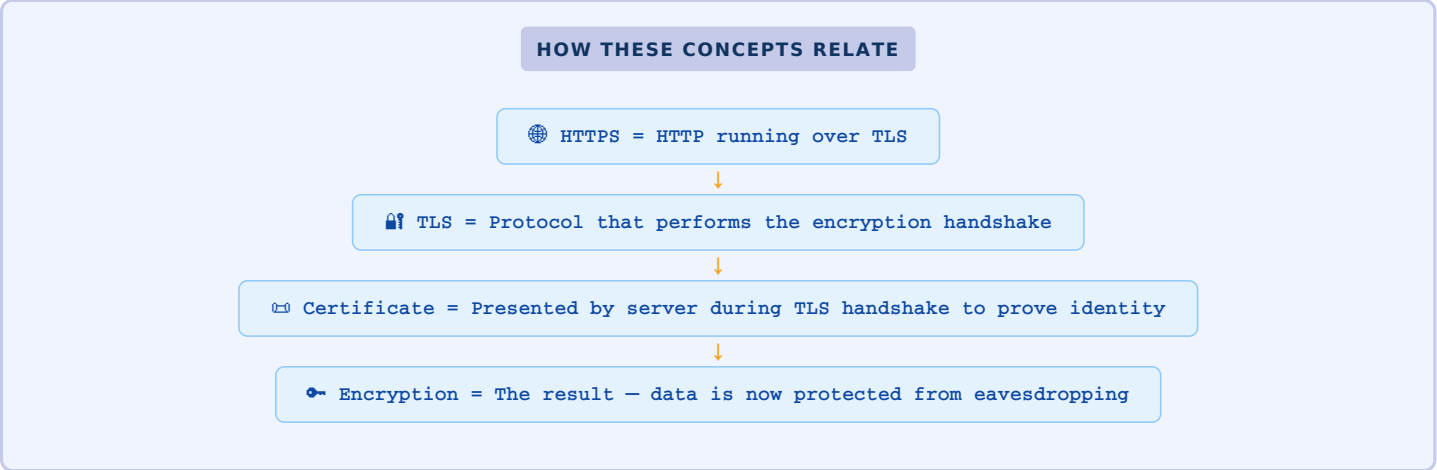
Aspect	HTTP	HTTPS
Full Name	HyperText Transfer Protocol	HTTP Secure
Port	80	443
Encryption	No — data sent in plaintext	Yes — data encrypted via TLS
Authentication	No server verification	Server identity verified via certificate
Data Integrity	Data can be tampered in transit	Data integrity guaranteed by TLS MAC
Certificate Required	No	Yes — SSL/TLS certificate
Browser indicator	⚠ Not Secure	Padlock icon
Use case	Avoid for any real use	All production web applications

SSL vs TLS

Aspect	SSL	TLS
Status	Deprecated, insecure	Current standard
Versions	SSL 2.0, 3.0	TLS 1.0, 1.1, 1.2, 1.3
Security	Vulnerable (POODLE, BEAST attacks)	Secure with TLS 1.2+ (TLS 1.3 best)

Used Today?	No — disabled on modern servers	Yes — TLS 1.2 and 1.3
Common usage	Term still used colloquially for TLS	Actual protocol used

Relationship: Certificate → TLS → HTTPS → Encryption



2.3 Certificate Types & Lifecycle

Public vs Private Certificates

Aspect	Public Certificate	Private Certificate
Issued by	Public CA (ACM acts as CA or uses Amazon's CA)	Private CA (ACM Private CA)
Trusted by	All browsers & OS automatically	Only within your organization
Cost	Free via ACM	ACM Private CA has a cost
Use case	Public-facing websites, APIs	Internal microservices, private apps
Validation	DNS or Email validation required	No public validation needed
Example	www.mycompany.com (public website)	internal.api.mycompany.local

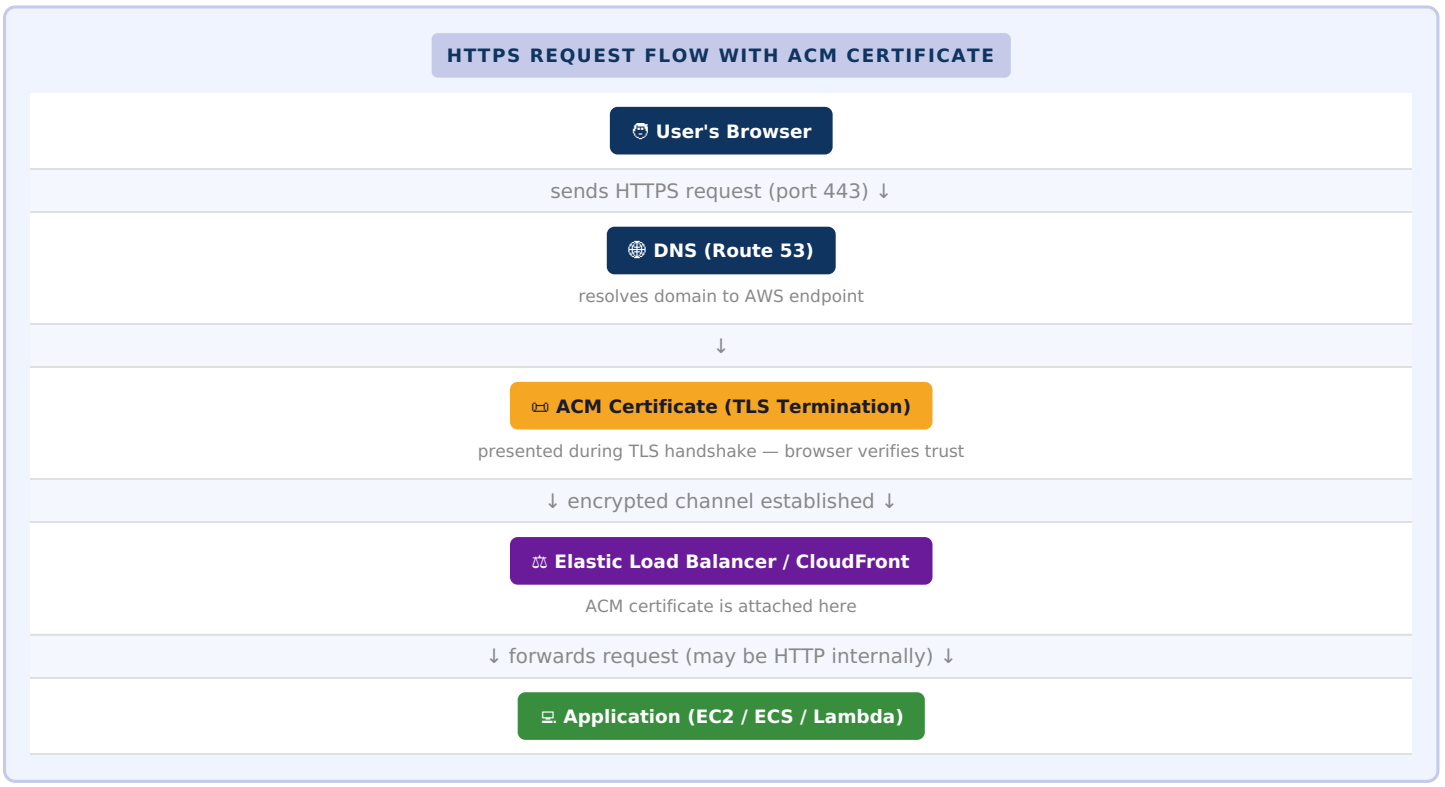
Certificate Lifecycle



Domain Validation Methods

DNS Validation ✓ Recommended	Email Validation
• ACM gives you a CNAME record to add to your DNS • ACM periodically checks for the CNAME • Supports **automatic renewal** — no manual action needed • Best for when you control the DNS	• ACM sends a validation email to domain admin contacts • Admin must click approval link in the email • Requires **manual action on each renewal** • Use when you don't control DNS

2.4 ACM Architecture & Request Flow



Architecture Explanation

- **TLS Termination at LB/CloudFront:** The ACM certificate is installed at the load balancer or CloudFront — not on the application itself
- **Encrypted externally:** All traffic between users and the load balancer is encrypted
- **Internal traffic:** Traffic between load balancer and app servers may be HTTP (within AWS's private network) or HTTPS (for end-to-end encryption)
- **ACM manages the certificate:** The private key of the certificate never leaves AWS — it's stored and managed by ACM

2.5 Real-World Example: HTTPS for a Web Application

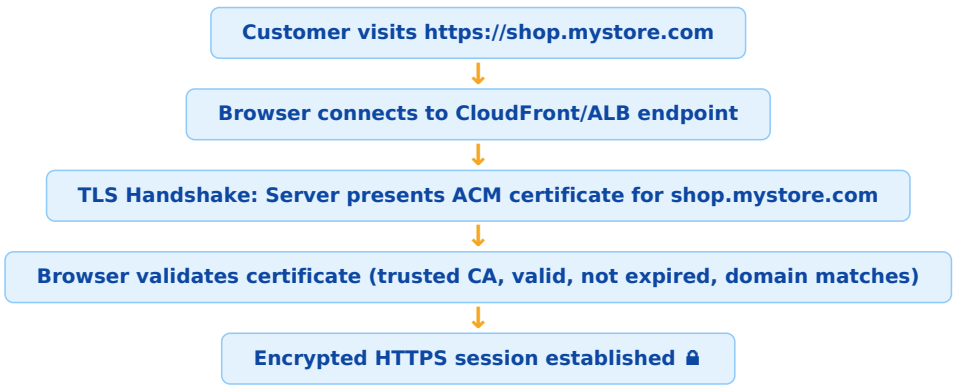
SCENARIO

You are launching an e-commerce website at *shop.mystore.com*. Customers must have a secure, encrypted connection to trust the site and submit payment info.

Why ACM Is Useful Here

- Without a certificate, the browser shows "Not Secure" — customers won't trust the site
- Without HTTPS, payment data travels in plaintext — major security risk
- Buying and managing certificates from a commercial CA is costly and requires annual renewal
- ACM provides a **free, auto-renewing certificate** for *shop.mystore.com*

Conceptual Flow



↓

Customer safely enters payment information

ACM Advantages

- Free public certificates for AWS-integrated services
- Automatic renewal — no expiry surprises
- Integrated with ALB, CloudFront, API Gateway, AppSync — one-click deployment
- Private key never exported from AWS — highly secure
- Supports wildcard certificates (*.mystore.com)

ACM Limitations

- ACM certificates **cannot be downloaded / exported** (private key stays in AWS)
- Only usable with supported AWS services — cannot install on a self-managed server
- Private certificates (ACM Private CA) have an associated cost
- Only available for domains you can validate (DNS or email)

ACM Terminology

CA (Certificate Authority)
Trusted organization that issues digital certificates and vouches for domain ownership

TLS Handshake
The initial negotiation process where client and server agree on encryption parameters and exchange certificate

Domain Validation (DV)
Certificate type that only verifies domain ownership (ACM provides DV certificates)

Wildcard Certificate
A certificate valid for all subdomains — e.g., *.mystore.com covers shop.mystore.com, api.mystore.com

SANs
Subject Alternative Names — a single certificate that covers multiple different domain names

Encryption in Transit
Data encrypted while traveling across the network — this is what TLS/HTTPS provides

TOPIC 3 OF 3

AWS Key Management Service (KMS)

Create, manage, and control cryptographic keys to encrypt your data across AWS

3.1 Definition & Purpose

DEFINITION

AWS Key Management Service (KMS) is a fully managed service that enables you to create, manage, and use *cryptographic keys* to encrypt and decrypt data. KMS integrates deeply with other AWS services to protect data at rest.

Core Concepts

🔐 Encryption
Transforming readable data (plaintext) into an unreadable format (ciphertext) using a cryptographic algorithm and key. Only those with the correct key can decrypt it.

🗝 Cryptographic Key
A piece of data (a string of bits) used by an algorithm to encrypt or decrypt information. The security of encryption depends on protecting the key.

🔧 Key Management
The full lifecycle management of keys: creation, storage, rotation, access control, auditing, and deletion.

🔐 KMS Key (CMK)
Customer Master Key — the primary resource in KMS. A logical key that KMS manages securely. Never leaves KMS in plaintext.

3.2 Encryption at Rest vs Encryption in Transit

📌 CRITICAL DISTINCTION

These are two different types of encryption for different threat scenarios. AWS uses KMS for at-rest; TLS/ACM for in-transit.

Aspect	Encryption at Rest	Encryption in Transit
What	Data stored on disk / database is encrypted	Data moving over network is encrypted
Threat Protected	Unauthorized physical/storage access	Network eavesdropping, man-in-the-middle
When Active	When data is stored and not being accessed	During network transmission
AWS Service	AWS KMS (with S3, EBS, RDS…)	ACM / TLS / HTTPS
Protocol/Standard	AES-256 typically	TLS 1.2 / TLS 1.3
Example	RDS database files on disk are encrypted	HTTPS connection to your web API
Key used by	KMS key (symmetric encryption)	TLS session keys (derived from certificate)

BEST PRACTICE:
Use *both* — encrypt data at rest
AND
in transit for complete protection. A strongly encrypted database accessed over plain HTTP is still vulnerable.

3.3 KMS Keys — Types & Management

KMS Key Types

Key Type	Created By	Managed By	Rotation	Cost	Control Level
AWS Managed Key	AWS	AWS	Automatic (yearly)	Free	Limited — cannot customize

	automatically				policy
Customer Managed Key (CMK)	You	You	Optional (enable/disable)	$1/month	Full — custom key policies, grants
AWS Owned Key	AWS (internal)	AWS	AWS managed	Free	None — invisible to you

Key Policies

- Every KMS key has a **key policy** — a resource-based policy that controls who can use and manage the key
- Key policies are the *primary* way to control access to KMS keys
- IAM policies can supplement key policies but cannot override them
- Key policies specify: key administrators (who manages), key users (who encrypts/decrypts)

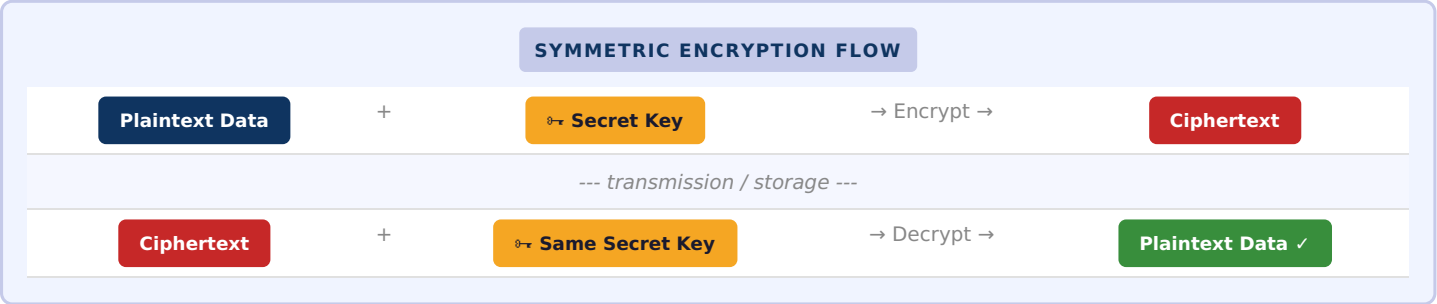
Key Rotation

- The practice of periodically generating new key material while keeping the same KMS key ID
- Old key material is retained to decrypt data encrypted with it
- New data is encrypted with the new key material
- AWS Managed Keys rotate automatically every year
- Customer Managed Keys: rotation must be explicitly enabled (annual automatic rotation)
- **Why rotate?** Limits the amount of data encrypted with any single key version — limits exposure if a key is compromised

3.4 Symmetric Encryption

DEFINITION

Symmetric encryption uses the *same single key* for both encryption and decryption. The sender and receiver must both possess the same secret key.



Advantages	Limitations
• Very fast — computationally efficient • Suitable for encrypting large amounts of data • Simple implementation • Standard: AES-256 is extremely secure	• Key distribution problem — how do you securely share the secret key? • If the key is intercepted, all encrypted data is compromised • Doesn't work well across untrusted channels

AWS KMS & SYMMETRIC KEYS:

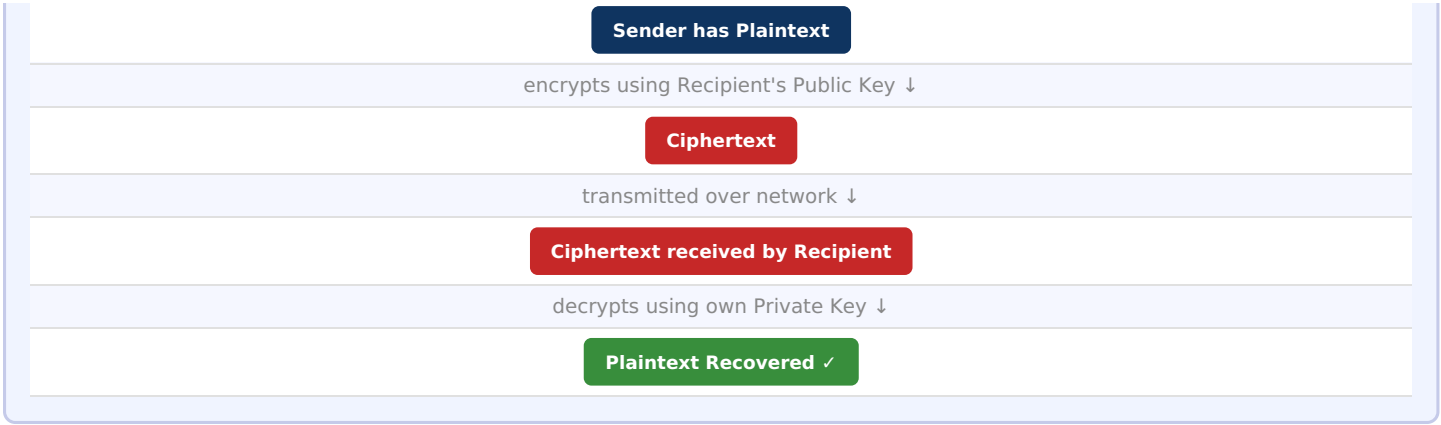
By default, KMS keys are symmetric (AES-256-GCM). When you call *kms:Encrypt*, KMS uses the symmetric key internally — the key never leaves KMS in plaintext.

3.5 Asymmetric Encryption

DEFINITION

Asymmetric encryption uses a *mathematically linked key pair*: a **public key** (can be shared freely) and a **private key** (kept secret). What one key encrypts, only the other can decrypt.





Digital Signatures (Asymmetric in reverse)

- Sender signs data using their **private key**
- Anyone with the sender's **public key** can verify the signature
- Proves: *the sender really sent this* (authenticity) and *the data wasn't changed* (integrity)
- Used in TLS certificates — the CA signs the certificate with its private key; browsers verify with CA's public key

Advantages	Limitations
• Solves key distribution — public key can be shared openly • Enables digital signatures for non-repudiation • Basis for TLS/HTTPS certificate trust • Secure key exchange over untrusted channels	• Computationally expensive — much slower than symmetric • Not suitable for encrypting large data directly • Key management complexity (public/private key pairs)

Symmetric vs Asymmetric — Full Comparison

Aspect	Symmetric	Asymmetric
Keys	One shared secret key	Key pair (public + private)
Speed	Fast (ideal for large data)	Slow (limited to small data)
Key sharing	Difficult — key must be kept secret	Easy — public key shared freely
Use in AWS	KMS data encryption (AES-256)	TLS handshake, digital signatures
Algorithm examples	AES-128, AES-256	RSA, ECC
Best for	Bulk data encryption	Key exchange, authentication, signatures
KMS support	Default KMS key type	Available as asymmetric KMS key

IN PRACTICE:
TLS (HTTPS) uses BOTH — asymmetric encryption during the handshake to securely exchange a symmetric session key, then uses that symmetric key for the actual data encryption. Best of both worlds.

3.6 Envelope Encryption ⭐ Important Concept

⭐ CORE KMS CONCEPT

Envelope encryption is how AWS services encrypt large amounts of data using KMS without sending all data through the KMS API — which would be slow and expensive.

DEFINITION

Envelope Encryption is a technique where data is encrypted with a *Data Encryption Key (DEK)*, and that DEK is itself encrypted with a *KMS Key* (master key). The encrypted DEK is stored alongside the encrypted data — like sealing a letter in an envelope.

Why Envelope Encryption?

- KMS API has a limit — you can only encrypt up to **4 KB** of data directly via `kms:Encrypt`
- Real data (files, database records, videos) is much larger than 4 KB
- Sending all data to KMS API would be: very slow, expensive, and bandwidth-intensive
- Solution: Generate a local DEK, use it to encrypt data locally, then use KMS only to protect the DEK





Key Terms in Envelope Encryption

Term	Full Name	Role
KMS Key	Key Encryption Key (KEK) / Master Key	Encrypts and decrypts the DEK. Never leaves KMS. Used minimally.
DEK	Data Encryption Key	Locally generated; encrypts the actual data. Used per-object. Discarded after use.
Ciphertext	Encrypted Data	The encrypted version of your actual data, produced by the DEK
Encrypted	Wrapped Data Key	The DEK encrypted by the KMS Key — stored alongside the encrypted data

DEK

WHY IT'S CALLED "ENVELOPE":

The plaintext data is encrypted (put in an envelope) with the DEK. That envelope (encrypted data) is stored with the encrypted DEK (the key is "in the envelope"). KMS only ever handles the small DEK, not the large data.

3.7 KMS Integration with AWS Services

Amazon S3 + KMS

WHAT GETS ENCRYPTED

- Object data (files, images, documents) stored in S3 buckets
- Each object can be encrypted with its own DEK

ENCRYPTION FLOW

- You upload a file → S3 requests a DEK from KMS → S3 encrypts the file locally with the DEK → stores encrypted file + encrypted DEK in S3 → plaintext DEK discarded
- On download: S3 retrieves encrypted DEK → asks KMS to decrypt it → uses DEK to decrypt file → returns plaintext file to you

OPTIONS:

SSE-S3 (S3 managed), SSE-KMS (KMS managed — more control), SSE-C (customer provided keys)

Amazon EBS + KMS

WHAT GETS ENCRYPTED

- EBS volume data — the entire block storage volume is encrypted
- Snapshots of encrypted volumes are also encrypted

ENCRYPTION FLOW

- Create encrypted EBS volume → specify KMS key → AWS generates a volume encryption key (DEK) encrypted by KMS key → all data written to disk is encrypted at the block level → EC2 reads/writes transparently (decryption happens in the hypervisor)

KEY POINT:

EC2 instance is unaware of encryption — it reads plaintext; encryption/decryption is handled transparently by the infrastructure

Amazon RDS + KMS

WHAT GETS ENCRYPTED

- Database storage files (data files, logs, backups, snapshots)
- Encryption is at the storage layer — not the application layer

ENCRYPTION FLOW

- Create RDS instance with encryption enabled → specify KMS key → RDS uses envelope encryption → all storage I/O is encrypted → automated backups and snapshots are also encrypted

IMPORTANT:

Encryption must be enabled at RDS instance creation — cannot encrypt an existing unencrypted instance directly (must snapshot → copy with encryption → restore)

KMS Advantages

- Centralized key management — one service for all your encryption keys
- Audit trail via CloudTrail — every KMS API call is logged
- FIPS 140-2 validated hardware security modules (HSMs) — government-grade security
- Deep integration with 100+ AWS services
- Fine-grained access control via key policies + IAM
- Key rotation support

KMS Limitations

- Customer managed keys cost $1/month per key + API call charges
- KMS API rate limits — high-throughput applications may hit throttling
- Keys are regional — a key in us-east-1 cannot decrypt data in eu-west-1 (use multi-region keys for this)
- KMS is not a secrets manager — use AWS Secrets Manager for database passwords and API keys

Related Services

Service	Relationship to KMS
AWS CloudTrail	Logs all KMS API calls — who used which key, when, from where
AWS Secrets Manager	Stores and rotates secrets (passwords, API keys); uses KMS to encrypt the stored secrets
AWS CloudHSM	Dedicated hardware security module for highest compliance; customer-managed HSMs. KMS can use CloudHSM as its key store.
AWS IAM	Controls who can make KMS API calls (e.g., kms:Decrypt); works with key policies
ACM	Uses KMS-managed key storage internally for certificate private keys

REFERENCE TABLES

Important Comparisons — All 15 Tables

Side-by-side comparison of all key concepts from IAM, ACM, and KMS

Table 1: IAM User vs IAM Role

Aspect	IAM User	IAM Role
Represents	Specific person or application	Assumable identity for any trusted entity
Credentials	Long-term (password, access keys)	Temporary (expires automatically via STS)
Best For	Human users needing console access	AWS services, cross-account, federated access
Security Risk	Long-term keys can be leaked/stolen	Lower — credentials auto-expire
Key Rotation	Manual rotation needed	Automatic — new temp credentials each time
Can be in Groups	Yes	No
Trust Policy	No	Yes — defines who can assume it
Use on EC2	Requires embedding access keys (bad)	Attach role to EC2 (best practice)

Table 2: IAM User vs IAM Group

Aspect	IAM User	IAM Group
What It Is	Individual identity	Collection of users
Has Credentials	Yes (password, access keys)	No
Can Log In	Yes	No
Policies Attached	Yes (directly or via group)	Yes (apply to all members)
Nesting	Can be in multiple groups	Cannot contain other groups
Purpose	Authenticate and act	Organize users for permission management

Table 3: IAM Role vs IAM Policy

Aspect	IAM Role	IAM Policy
What It Is	An identity that can be assumed	A document defining permissions
Format	AWS resource with ARN	JSON document
Relationship	Has policies attached to it	Attached to users, groups, or roles
Independently Exists	Yes	Yes (managed) or No (inline)
Does Work	Role enables you to act	Policy defines what actions are permitted

Table 4: Authentication vs Authorization

Aspect	Authentication	Authorization
Question	Who are you?	What can you do?
Purpose	Verify identity	Grant/deny access to resources
Occurs	First (before authorization)	After successful authentication
AWS mechanism	Password, MFA, access keys	IAM policies & permission evaluation
Failure response	Login rejected / credentials invalid	403 Access Denied
Analogy	Security guard checks your ID	ID badge only opens certain doors

Table 5: Identity-based vs Resource-based Policy

Aspect	Identity-Based Policy	Resource-Based Policy
Attached To	IAM User, Group, or Role	AWS Resource (S3 bucket, KMS key, SQS queue)
Controls	What the identity can do	Who can access the resource
Principal Field	Implied (the attached identity)	Explicit (specifies who)
Cross-account	Needs role assumption	Can directly grant cross-account access
Most common	Yes — standard permission management	Less common — used for specific sharing scenarios

Table 6: AWS Managed vs Customer Managed Policy

Aspect	AWS Managed Policy	Customer Managed Policy
Creator	AWS	You
Updates	AWS updates automatically	You update manually
Customization	None — read-only	Full — modify as needed
Versioning	AWS manages versions	You manage versions
Reusability	Any AWS account worldwide	Within your AWS account
Best for	Common standard permissions	Precise, organization-specific access
Example	AmazonS3ReadOnlyAccess	MyAppDatabasePolicy

Table 7: Managed vs Inline Policy

Aspect	Managed Policy	Inline Policy
Existence	Standalone — has its own ARN	Embedded inside user/group/role
Reusability	Attach to multiple identities	Only for one specific identity
Versioning	Supported (up to 5 versions)	Not supported
Visibility	Visible in IAM policies list	Only visible within the identity
Deletion	Persists after detaching	Deleted when the identity is deleted
Best for	Shared, reusable permissions	Strict 1:1 binding that must not be shared

Table 8: ACM vs KMS

Aspect	AWS ACM	AWS KMS

Primary Purpose	SSL/TLS certificate management	Encryption key management
Protects	Data in transit (network encryption)	Data at rest (storage encryption)
Key Type	Asymmetric (RSA/ECC certificate keys)	Symmetric (AES-256) or asymmetric
Used With	ALB, CloudFront, API GW, Route 53	S3, EBS, RDS, Lambda, DynamoDB…
Certificates	Yes — issues and manages them	No — manages encryption keys, not certs
Cost	Free for public certificates	$1/month per CMK + API calls
Related Protocol	TLS / HTTPS	AES, RSA (algorithm-level)
Overlap	ACM uses KMS-backed storage for private keys	KMS can manage asymmetric keys used in signing

Table 9: Encryption vs Hashing

Aspect	Encryption	Hashing
Reversible	Yes — with the correct key	No — one-way function
Output	Ciphertext (variable size)	Fixed-length hash digest
Key Required	Yes — to encrypt and decrypt	No key — deterministic function
Purpose	Confidentiality — protect data	Integrity — verify data hasn't changed
Use cases	S3, EBS, RDS, HTTPS	Password storage, file integrity, digital signatures
Algorithm examples	AES-256, RSA	SHA-256, SHA-3, bcrypt
AWS service	KMS, ACM	Used internally in many AWS services

Table 10: Encryption at Rest vs Encryption in Transit

Aspect	At Rest	In Transit
When	Data stored on disk/DB	Data moving across network
Threat	Physical theft, unauthorized storage access	Network eavesdropping, MITM attacks
AWS Service	KMS (with S3, EBS, RDS...)	ACM / TLS
Protocol	AES-256 (symmetric)	TLS 1.2 / 1.3
Example	RDS database files encrypted on disk	HTTPS browser-to-server connection

Table 11: Symmetric vs Asymmetric Encryption

Aspect	Symmetric	Asymmetric
Keys Used	One secret key (shared)	Two keys: public + private
Speed	Fast	Slow
Data Size	Any size (bulk encryption)	Small data only
Key Exchange	Challenging — key must stay secret	Easy — share public key openly
Use in KMS	Default key type (AES-256)	Available for signing and specific use cases
Algorithm	AES-128, AES-256	RSA-2048, RSA-4096, ECC
TLS usage	Bulk data encryption after handshake	Key exchange and server authentication in handshake

Table 12: HTTP vs HTTPS

Aspect	HTTP	HTTPS
Full Form	HyperText Transfer Protocol	HTTP Secure
Port	80	443
Encryption	None (plaintext)	TLS encryption
Authentication	No server identity verification	Server verified via certificate
Data Integrity	Not guaranteed	Guaranteed via TLS MAC
Certificate	Not needed	Required (ACM provides it)
Browser display	⚠ Not Secure warning	🔒 Padlock — Secure
Use cases	Internal testing only	All production applications

Table 13: SSL vs TLS

Aspect	SSL	TLS
Status	Deprecated and insecure	Current standard
Versions	SSL 2.0, SSL 3.0	TLS 1.0, 1.1, 1.2, 1.3
Security	Vulnerable (POODLE, BEAST, DROWN)	Secure (TLS 1.2+ recommended)
In use today?	No — disabled on all modern servers	Yes — TLS 1.2 and TLS 1.3
Colloquial use	People still say "SSL certificate"	Actual protocol behind it is TLS
ACM provides	No SSL — TLS certificates only	Yes — TLS certificates

Table 14: Public vs Private Certificate

Aspect	Public Certificate	Private Certificate
Issued By	Public CA (trust included in OS/browser)	Private CA (ACM Private CA)
Trusted By	All browsers and devices globally	Only systems configured to trust your CA
Cost	Free via ACM (for integrated AWS services)	ACM Private CA charges apply
Use Case	Public websites, public APIs	Internal apps, microservices, private networks
Domain Validation	Required (DNS or Email)	Not publicly validated
Example	www.mycompany.com	internal.api.corp.local

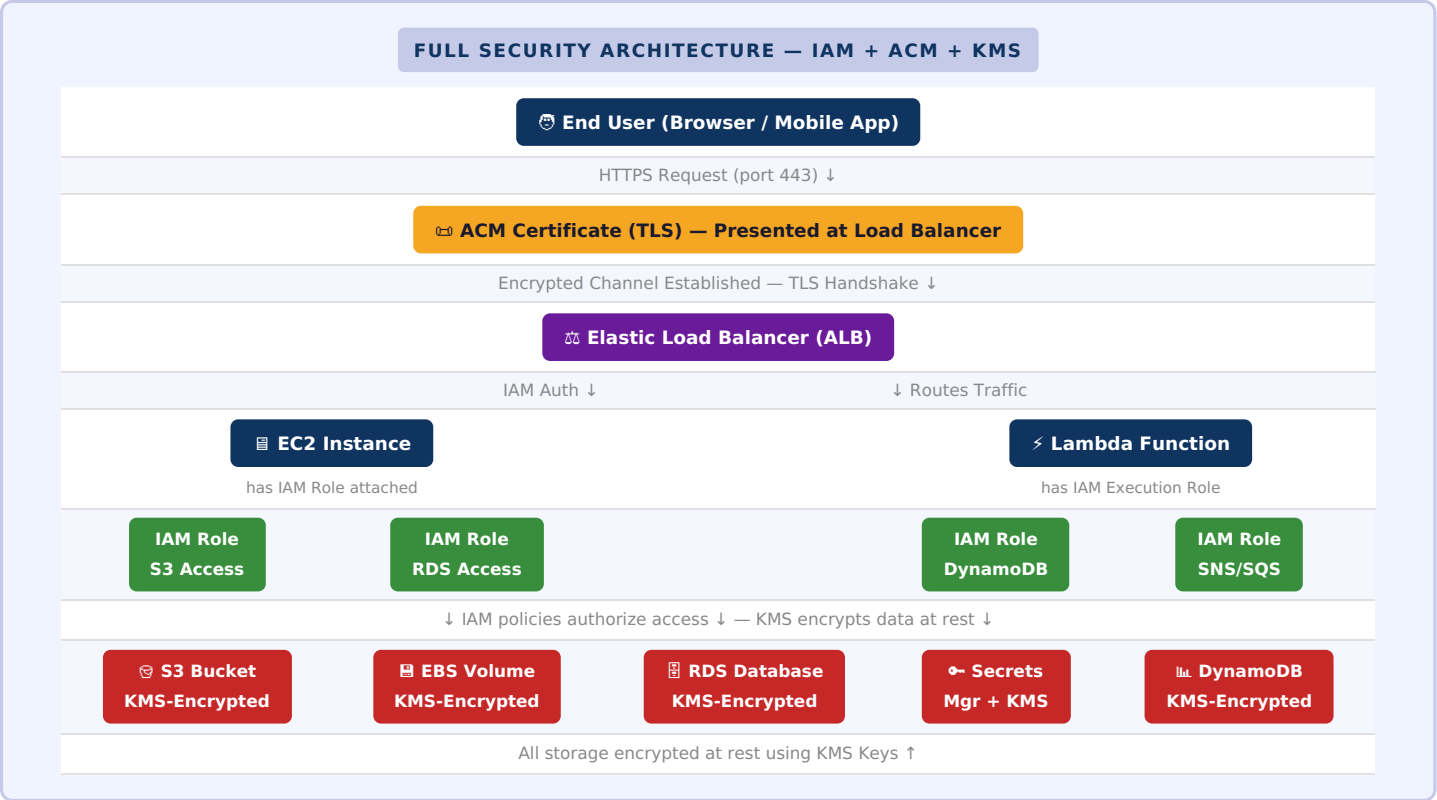
Table 15: AWS Managed Key vs Customer Managed Key

Aspect	AWS Managed Key	Customer Managed Key (CMK)
Created By	AWS automatically (per service)	You explicitly create it
Key ID Format	aws/service (e.g., aws/s3)	Custom alias you define
Key Policy	AWS controls — cannot customize	You define and manage key policy
Rotation	Automatic (yearly)	Configurable (optional, annual)
Cost	Free	$1/month + API call charges
Access Control	Limited	Full — define exactly who can use/manage
Cross-account use	Not shareable	Can grant cross-account access
Audit	CloudTrail logs usage	CloudTrail logs all usage (full audit)
Best for	Basic encryption needs, simple setup	Compliance, fine-grained control, custom rotation

INTEGRATED VIEW

Complete AWS Security Architecture

How IAM, ACM, and KMS work together to secure a complete AWS application



Architecture Component Explanation

Layer 1: User Access (ACM / TLS)

- All user traffic enters via **HTTPS only** — HTTP redirected to HTTPS
- **ACM certificate** attached to ALB or CloudFront terminates TLS
- TLS ensures *encryption in transit* — eavesdroppers cannot read traffic
- Certificate validates server identity — users can trust they're on the real site

Layer 2: Access Control (IAM)

- EC2 instances and Lambda functions have **IAM Roles** attached — no hardcoded credentials
- IAM Roles grant *only the permissions needed* for each component (least privilege)
- EC2 for the web tier: can read from S3 and query RDS — but cannot delete KMS keys
- Lambda for processing: can write to DynamoDB — but cannot access EC2
- IAM policies evaluated for every AWS API call made by any component
- Administrators access AWS using **IAM Users + MFA** — not root

Layer 3: Data Encryption (KMS)

- **S3 buckets** use SSE-KMS — each uploaded object encrypted with a DEK wrapped by KMS key
- **EBS volumes** — OS and application data encrypted at block level using KMS
- **RDS databases** — all database files, logs, and backups encrypted with KMS
- **Secrets Manager** — stores DB passwords encrypted with KMS, rotated automatically
- All encryption uses **envelope encryption** — KMS never directly handles large data
- CloudTrail logs every KMS decrypt/encrypt call — full audit trail

Security Best Practices Illustrated in This Architecture

Best Practice	How It's Implemented

Encrypt in transit	ACM certificate on ALB, HTTPS enforced
Encrypt at rest	KMS encryption on all storage services
Least privilege	Each EC2/Lambda role has minimal permissions
No root usage	Root account locked; IAM admin users for management
No hardcoded credentials	IAM Roles instead of access keys on EC2
MFA enforced	IAM users require MFA for console access
Audit trail	CloudTrail logs all IAM and KMS API calls
Key rotation	KMS customer managed keys have rotation enabled

FINAL SUMMARY

Concept Map — Complete Module Summary

Everything learned in one visual map — IAM, ACM, KMS and their relationships

🔑 IAM

```
IAM
├── Authentication
│   ├── Password
│   ├── Access Keys
│   └── MFA
├── Authorization
│   ├── Policy Evaluation
│   ├── Users
│   │   ├── Console Password
│   │   └── Access Keys
│   ├── Groups
│   │   └── Permission Grouping
│   ├── Roles
│   │   ├── Trust Policy
│   │   ├── Temp Credentials
│   │   └── STS
│   └── Policies
│       ├── Identity-based
│       ├── Resource-based
│       ├── AWS Managed
│       ├── Customer Managed
│       └── Inline
└── Principles
    ├── Least Privilege
    └── Root Protection
```

🔒 ACM

```
ACM
├── Certificates
│   ├── Public Certs (free)
│   └── Private Certs (paid)
├── Protocols
│   ├── TLS 1.2 / 1.3
│   └── HTTPS
├── Validation
│   ├── DNS Validation
│   └── Email Validation
├── Lifecycle
│   ├── Request
│   ├── Validate
│   ├── Issue
│   ├── Deploy
│   └── Auto-Renew
├── Integrations
│   ├── ALB
│   ├── CloudFront
│   └── API Gateway
└── Security Goal
    └── Encryption in Transit
```

🗝️ KMS

```
KMS
├── Encryption Types
│   ├── Symmetric (default)
│   └── Asymmetric
├── Keys
│   ├── AWS Managed
│   └── Customer Managed
├── Key Policies
│   ├── Key Admins
│   └── Key Users
├── Key Rotation
│   ├── Automatic (annual)
│   └── Old material kept
├── Envelope Encryption
│   ├── DEK (Data Key)
│   ├── KMS Key wraps DEK
│   └── Local encryption
└── Integrations
    ├── S3 SSE-KMS
    ├── EBS Encryption
    └── RDS Encryption
```

Concept Map Explained

- **IAM** is the foundation — it controls *who* can access *what*. Everything in AWS is accessed via IAM-authenticated and authorized calls. Authentication proves identity; authorization (via policies) determines permissions. Users are for humans, Groups simplify management, Roles enable services to act securely without static credentials, and Policies define the rules.
- **ACM** solves the *encryption in transit* problem — ensuring data traveling between users and AWS services is encrypted and the server's identity is verified. It does this by managing the full lifecycle of TLS certificates, integrating with AWS load balancers and CDN, and automatically renewing them.
- **KMS** solves the *encryption at rest* problem — ensuring stored data is unreadable without the correct key. It manages cryptographic keys securely (keys never leave KMS in plaintext), integrates with storage services (S3, EBS, RDS), and uses envelope encryption to handle large data efficiently without performance penalties.
- **Together:** IAM controls access, ACM secures the network channel, and KMS protects the stored data — a complete three-layer security model for AWS applications.

📌 **THE THREE QUESTIONS OF AWS SECURITY**

- **WHO ARE YOU?**
  → `IAM Authentication` (username, password, MFA, access keys)
- **WHAT CAN YOU DO?**
  → `IAM Authorization` (policies, roles, permissions)
- **IS THE CONNECTION AND DATA SAFE?**
  → `ACM` (in-transit) + `KMS` (at-rest)